

Finish Form Validation

We have our form submitting data to the store method in the `JobController`. We need to add the fields to the store method for validation and submission.

Let's open the `store` method in the `App\Http\Controllers\JobController` class. Right now it looks like this:

```
public function store(Request $request): RedirectResponse
{
    // Validate the incoming request data
    $validatedData = $request->validate([
        'title' => 'required|string|max:255',
        'description' => 'required|string',
    ]);

    // Create a new job listing with the validated data
    Job::create([
        'title' => $validatedData['title'],
        'description' => $validatedData['description'],
    ]);

    return redirect()->route('jobs.index');
}
```

This is ok if we only want a title and a description, but we have tons more fields now, so we need to handle those. Let's update the `store` method to handle all of the new fields. Here is the updated `store` method:

```
public function store(Request $request): RedirectResponse
{
    // Validate the incoming request data
    $validatedData = $request->validate([
        'title' => 'required|string|max:255',
        'description' => 'required|string',
        'salary' => 'required|integer',
        'tags' => 'nullable|string',
        'job_type' => 'required|string',
        'remote' => 'required|boolean',
        'requirements' => 'nullable|string',
        'benefits' => 'nullable|string',
        'address' => 'nullable|string',
        'city' => 'required|string',
        'state' => 'required|string',
        'zipcode' => 'required|string',
        'contact_email' => 'required|email',
        'contact_phone' => 'nullable|string',
        'company_name' => 'required|string',
        'company_description' => 'nullable|string',
    ]);

    // Create a new job listing with the validated data
    Job::create([
        'title' => $validatedData['title'],
        'description' => $validatedData['description'],
        'salary' => $validatedData['salary'],
        'tags' => $validatedData['tags'],
        'job_type' => $validatedData['job_type'],
        'remote' => $validatedData['remote'],
        'requirements' => $validatedData['requirements'],
        'benefits' => $validatedData['benefits'],
        'address' => $validatedData['address'],
        'city' => $validatedData['city'],
        'state' => $validatedData['state'],
        'zipcode' => $validatedData['zipcode'],
        'contact_email' => $validatedData['contact_email'],
        'contact_phone' => $validatedData['contact_phone'],
        'company_name' => $validatedData['company_name'],
        'company_description' => $validatedData['company_description'],
    ]);

    return redirect()->route('jobs.index');
}
```

```
        'company_logo' => 'nullable|image|mimes:jpeg,png,jpg,gif|max:2048',  
        'company_website' => 'nullable|url',  
    ]);  
  
    // Create a new job listing with the validated data  
    Job::create($validatedData);  
  
    return redirect()->route('jobs.index');  
}
```

Handle User ID

Another issue is that ultimately, all listings will be connected to a user ID but we do not have authentication yet. So for now, let's hard code the user ID to 1.

Add this line under the `$validatedData` variable:

```
// Add the hardcoded user_id  
$validatedData['user_id'] = 1;
```

Send Success Message

When we redirect, we can attach a success message to the session:

```
return redirect()->route('jobs.index')->with('success', 'Job listing created  
successfully!');
```

We will have to handle this in the view. We will do that in the next lesson.

Now if you try and submit an empty form, you will get all kinds of errors. If you fill out the form correctly, you will be redirected to the jobs index page. From this point on, I would suggest using a browser extension such as Fake Filler to fill out the form. Otherwise, you will need to manually fill out the form and it's a long form.

We are now able to store all of the new fields in the database.